

Assignment: ETL to Insights

Duration: 2 weeks

Requirements

Design and implement a complete end-to-end ETL pipeline for the provided dataset, including analytics generation and data visualization.

Dataset: employee, timesheet

ETL Pipeline

- 1. Extract and Load:**
 - Read and load the provided files into the database.
 - With support for loading data either directly from local files and from MinIO (S3-compatible) storage.
- 2. Transform:**
 - Clean the data (handle missing values, correct data types, remove duplicates, trim extra spaces)
 - Create derived columns for key business metrics
- 3. Post-Processing:**
 - Populate any subsequent tables or derived datasets
 - Run quality control checks and validation scripts
 - Execute report generation scripts automatically
- 4. Orchestration:**
 - Create a flow-based program to automate the ETL process end-to-end.
 - Define task dependencies and ensure proper execution order.

Analytics with SQL

Once the dataset is loaded, explore it to understand its structure and identify potential business insights. Based on your findings, write SQL queries to produce the following insights (Which must include but not be limited to):

KPI Name

Definition

Purpose / Insight

Active Headcount Over Time	Number of employees actively employed on a given date, based on hire and termination records.	Helps track workforce size and identify hiring or attrition trends over time.
Turnover Trend	Measure of employee terminations across specific time periods (e.g., monthly or quarterly).	Monitors organizational stability and highlights peak turnover periods.
Average Tenure by Department	Average employment duration of staff within each department.	Evaluates retention effectiveness and workforce experience per department.
Average Working Hours per Employee	Mean number of hours worked per day or per week by each employee.	Indicates productivity levels and workload balance.
Late Arrival Frequency	Number of times an employee clocked in later than the scheduled start time(Consider grace time +/- 5 min).	Assesses punctuality and discipline across employees or teams.
Early Departure Count	Number of days employees left earlier than the expected shift end time(Consider grace time +/- 5 min).	Highlights attendance irregularities and potential productivity loss.
Total Overtime Count	Total number of workdays or hours where employees exceeded standard shift duration(Consider grace time +/- 5 min).	Highlights workload pressure, potential fatigue, and need for workforce optimization.
Rolling Average Working Hours	Moving average of working hours across a defined recent time window.	Detects trends such as increasing overtime or reduced productivity.
Early Attrition Rate	Proportion of employees who leave within the first few months of joining.	Identifies issues with recruitment, onboarding, or job satisfaction.

API

Design and implement a simple RESTful API that allows basic operations on the employee table and read-only access to the timesheet table.

1. Employee API (CRUD)

- **Create:** Add a new employee with relevant details (e.g., id, name, department, role, date_joined).
- **Read:** Retrieve employee details by id or list all employees.
- **Update:** Modify existing employee information.
- **Delete:** Remove an employee from the database.

2. Timesheet API (Read-only)

- Retrieve timesheet entries for a given employee.
- List all timesheets, filterable by date range or employee.

The API should also include authentication and authorization to ensure that only authorized users can access or modify data.

Visualization

- Create at least three visualizations from your analytics results using any tool of your choice (Python libraries, Power BI, Tableau, Excel, etc.)
- Focus on creating clear, actionable insights that tell a data story
- Include interactive elements where appropriate (filters, drill-downs, time ranges)

Tech Stack

- **ETL:** Python (Pandas, SQLAlchemy, or any preferred libraries) with orchestration (Luigi or any other library)
- **API:** Python (FastAPI or Flask)
- **Database:** Any relational database (PostgreSQL, MySQL, SQLite, DuckDB, etc.)
- **Visualization:** Any visualization tool or library of your choice

NOTE: You may use any frameworks or libraries to assist your work, but avoid fully managed third-party ETL/BI tools that hide implementation details. You are expected to understand and justify engineering decisions you make.

Expectations

Core Expectations

The final deliverable should be usable and demonstrate:

ETL Implementation

- Python code that extracts, transforms, and loads data into a database.
- Include orchestration to define and manage task dependencies.
- Database connections, file paths, and processing options should be configurable via environment files or YAML/JSON files.
- Code should be modular, readable, and well-documented.
- Implement comprehensive error handling, logging, and support for reruns on task failures.

Database Design

- Appropriately-normalized schema with relationships and constraints
- Follow industry-standard data modelling patterns (e.g., Medallion architecture, dimensional modeling, etc)
- Data validation rules to ensure integrity
- Proper indexing strategy for query performance optimization

API Design

- Full CRUD for employees and read-only access for timesheets.
- Authentication and authorization for all endpoints.
- Modular, readable, and well-documented code.
- Proper error handling, logging, and RESTful design.
- Optional: API documentation (Swagger/OpenAPI) and input validation.

Analytics Queries

- SQL queries that produce correct and meaningful business insights
- Proper use of joins, aggregations, window functions, and filtering
- Optimized queries with clear documentation and comments

Visualizations

- Clear, accurate, and insightful visualizations that tell a data story
- Interactive elements where possible (filters, drill-downs)
- Proper labeling, legends, and formatting

Additional Features to Consider

- **Database Migration:** Use of database migrations for schema management
 - **Data Quality Validation:** Implement automated data quality checks with configurable rules and generate quality reports
 - **Containerization:** Dockerize the entire pipeline with docker for easy deployment and environment consistency
-

Documentation

Include a doc file that clearly explains:

- **Engineering Decisions:** Technology choices, architecture decisions, and trade-offs made
- **Setup Instructions:** Step-by-step guide to setup the pipeline
- **Usage Guide:** How to run the ETL pipeline and generate reports
- **Schema Documentation:** Database design with entity relationships